Esercizio Laboratorio 3:

Sviluppare in linguaggio C una soluzione software distribuita che permetta a dei Sensori di Traffico di inviare dati a un Server Centrale. Il Server elabora questi dati e, se si verificano condizioni critiche di traffico, inoltra un allarme a delle Centrali di Polizia connesse.

Il sistema è composto da tre entità:

1. Sensore di Traffico (Traffic Sensor Node)

- **Protocollo:** La comunicazione verso il Server Centrale deve essere **NON affidabile (UDP)**.
- **Comportamento:** * Simula la lettura dei dati inviando un messaggio ogni 3 secondi.
  - I dati inviati contengono: `ID_Sensore` (intero a 16 bit), `numero_veicoli` (intero a 32 bit), `velocita_media` (float a 32 bit).
  - (Suggerimento: i dati generati possono essere randomici o inseriti da riga di comando per testare l'allarme).

2. Server Centrale (Main Server)

- **Protocollo:** Deve gestire contemporaneamente connessioni **UDP** (per i sensori) e **TCP** (per le Centrali di Polizia).
- **Comportamento:**
  - **Ricezione Dati:** Ascolta sulla porta UDP i dati in arrivo dai vari sensori e aggiorna in memoria l'ultimo stato di ogni sensore.
  - **Allarmi:** Se riceve un dato in cui il `numero_veicoli` > 50 oppure la `velocita_media` < 15.0 km/h, il Server entra in stato di allarme per quel sensore.

- **Notifica TCP:** Non appena si genera un allarme, il Server deve inviare un messaggio di "TRAFFIC JAM ALERT" (con l'ID del sensore e la causa) a **tutte** le Centrali di Polizia attualmente connesse in TCP.
- **Persistenza:** Ogni allarme generato deve essere salvato in un database (un file di testo `log_allarmi.txt`). Al riavvio, il file non deve essere sovrascritto, ma bisogna accodare i nuovi log (modalità append).
- **Concorrenza:** Deve gestire la ricezione UDP e i vari client TCP contemporaneamente (tramite `pthread` o multiplexing con `select`/`poll`). Bisogna proteggere le strutture dati condivise (es. la lista dei client TCP connessi) con dei `mutex`.

3. Centrale di Polizia (Control Node)

- **Protocollo:** La comunicazione con il Server Centrale deve essere **affidabile (TCP)**.
- **Comportamento:**
  - Si connette al Server Centrale al momento dell'avvio.
  - Rimane perennemente in ascolto, stampando a video tutti gli allarmi di traffico che riceve dal Server Centrale in tempo reale.
  - **[Bonus]**: L'utente della Centrale può digitare da terminale il comando testuale `STATUS <ID>` per richiedere al Server l'ultimo stato noto di un sensore (es. `STATUS 1234`). Il server risponderà in TCP con gli ultimi dati registrati.

Requisiti Tecnici Richiesti:

1. **Definizione delle Struct:** Usa `#pragma pack(push, 1)` per definire la struttura del pacchetto UDP del sensore, evitando problemi di padding dell'architettura.

2. **Gestione dell'Endianness:** Assicurati che tutti i dati multi-byte scambiati in rete siano in **Network Byte Order**. Fai particolare attenzione alla serializzazione del tipo `float` per la velocità media (utilizza il trick della `memcpy` su un `uint32_t` visto a lezione prima di fare la `htonl`).
3. **Gestione delle disconnessioni:** Se una Centrale di Polizia si disconnette, il Server TCP deve accorgersene (`recv` ritorna 0) e rimuovere il descrittore del socket dalla lista per non tentare di inviargli futuri allarmi.